

API Specification

1. Purpose

The API layer provides a clear contract between the frontend and backend.

It should support:

- Public content access
- Authentication-aware actions
- Search
- Bookmarks
- Learning progress
- User preferences
- AI chat
- Admin content management
- Source and review workflows

The API should remain simple enough for the web MVP while being reusable by future mobile apps.

2. API Style

Recommended approach:

REST-style endpoints using Next.js Route Handlers

Use Server Actions for internal web-only workflows where appropriate.

Important mobile-ready operations should still have reusable service boundaries.

3. Base Path

Recommended API base:

```
/api
```

Example:

```
/api/search  
/api/ai/chat  
/api/bookmarks
```

4. Response Format

Recommended success response:

```
{  
  "success": true,  
  "data": {}  
}
```

Recommended error response:

```
{  
  "success": false,  
  "error": {  
    "code": "ERROR_CODE",  
    "message": "Human-readable message"  
  }  
}
```

5. HTTP Status Codes

Use standard codes:

- 200 Success
- 201 Created
- 204 Success with no response body
- 400 Invalid request
- 401 Authentication required
- 403 Not permitted
- 404 Not found
- 409 Conflict
- 422 Validation error
- 429 Too many requests
- 500 Server error

- `503` Service temporarily unavailable
-

6. Input Validation

All API inputs should be validated server-side.

Recommended:

Zod

Never trust request payloads directly.

7. Authentication

Authenticated requests should use the current Supabase user session.

Do not accept a `user_id` from the browser as proof of identity.

The backend should derive the authenticated user ID from the session.

8. Public Content Endpoints

GET `/api/scriptures`

Returns published scriptures.

Optional query parameters:

```
locale
type
limit
page
```

Example:

```
/api/scriptures?locale=en&type=upanishad
```

9. Get Scripture

GET `/api/scriptures/{slug}`

Returns:

- Scripture metadata
 - Localized description
 - Structure summary
 - Verification status
 - Available languages
 - Related content
-

10. Scripture Sections

GET `/api/scriptures/{slug}/sections`

Returns sections for a scripture.

Example:

Bhagavad Gita:

- Chapter 1
 - Chapter 2
 - ...
 - Chapter 18
-

11. Get Scripture Section

GET `/api/scriptures/{slug}/sections/{sectionSlug}`

Returns:

- Section metadata
 - Summary
 - Verse count
 - Key themes
 - Verse list
-

12. Verse Endpoint

GET `/api/verses/{id}`

Returns structured verse data.

Example response concept:

```
{
  "id": "...",
  "reference": "Bhagavad Gita 2.47",
  "sanskrit": "...",
  "transliterations": [],
  "translations": [],
  "commentaries": [],
  "explanations": [],
  "sources": [],
  "relatedContent": []
}
```

13. Verse by Canonical Reference

Optional convenience endpoint:

GET `/api/verses/lookup`

Query parameters:

```
scripture
chapter
verse
locale
```

Example:

```
/api/verses/lookup?scripture=bhagavad-gita&chapter=2&verse=47&locale=en
```

Useful for exact scripture navigation.

14. Concepts Endpoint

GET `/api/concepts`

Supports:

```
locale
difficulty
tag
page
limit
```

15. Concept Detail

GET `/api/concepts/{slug}`

Returns:

- Name
 - Sanskrit name
 - Simple explanation
 - Deep explanation
 - Sources
 - Related verses
 - Related concepts
 - Traditions where relevant
-

16. Deities Endpoint

GET `/api/deities`

Supports:

```
locale
limit
page
```

17. Deity Detail

GET `/api/deities/{slug}`

Returns:

- Overview
 - Sanskrit name
 - Iconography
 - Philosophy
 - Stories
 - Scriptures
 - Mantras
 - Festivals
 - Traditions
 - Sources
-

18. Mantras Endpoint

GET `/api/mantras`

Filters:

```
deity
type
locale
tag
```

19. Mantra Detail

GET `/api/mantras/{slug}`

Returns:

- Sanskrit text
- Transliteration
- Meaning
- Context
- Traditional usage

- Source
 - Associated deity
 - Related content
-

20. Festivals Endpoint

GET `/api/festivals`

Query parameters:

```
locale
year
month
region
upcoming
```

Example:

```
/api/festivals?year=2027&month=10&region=india
```

21. Festival Detail

GET `/api/festivals/{slug}`

Returns:

- Name
 - Description
 - Meaning
 - Practices
 - Regional variations
 - Relevant dates
 - Related deity
 - Related scripture
 - Sources
-

22. Learning Paths

GET `/api/learning-paths`

Filters:

```
locale
difficulty
```

23. Learning Path Detail

GET `/api/learning-paths/{slug}`

Returns:

- Title
 - Description
 - Difficulty
 - Modules
 - Estimated duration
 - User progress if authenticated
-

24. Search API

GET `/api/search`

Required parameter:

```
q
```

Optional:

```
locale
type
page
limit
```

Example:

```
/api/search?q=karma&locale=en
```

25. Search Response

Recommended response:

```
{
  "query": "karma",
  "results": [
    {
      "type": "concept",
      "id": "...",
      "title": "Karma",
      "snippet": "...",
      "url": "/en/learn/karma",
      "score": 0.92
    }
  ]
}
```

26. Search Types

Supported values:

```
all
concept
scripture
verse
deity
mantra
festival
learning_path
```

27. Search Suggestions

GET `/api/search/suggestions`

Example:

```
/api/search/suggestions?q=kar&locale=en
```

Returns:

- Exact titles
 - Aliases
 - Common related queries
-

28. Authentication Endpoints

Most authentication can use Supabase directly through secure integration.

Application-specific endpoints may include:

GET `/api/me`

Returns current user's profile and preferences.

29. Profile Update

PATCH `/api/me/profile`

Authenticated only.

Example request:

```
{
  "displayName": "Darshil",
  "preferredLanguage": "en"
}
```

30. Preferences

GET `/api/me/preferences`

Returns account preferences.

PATCH `/api/me/preferences`

Possible values:

- preferred language
 - theme
 - default commentary
 - reading mode
-

31. Bookmark Creation

POST `/api/bookmarks`

Authenticated only.

Request:

```
{
  "contentType": "verse",
  "contentId": "...",
}
```

32. Get Bookmarks

GET `/api/bookmarks`

Optional filters:

```
type
page
limit
```

33. Remove Bookmark

DELETE `/api/bookmarks/{id}`

Or idempotent alternative:

```
DELETE /api/bookmarks?contentType=verse&contentId=...
```

34. Reading History

POST `/api/history`

Authenticated.

Request:

```
{
  "contentType": "verse",
  "contentId": "...",
}
```

The backend should upsert rather than create unlimited duplicate rows.

35. Get Reading History

GET `/api/history`

Supports pagination and content type filter.

36. Learning Progress Update

POST `/api/progress`

Authenticated.

Request:

```
{
  "learningPathId": "...",
  "moduleId": "...",
  "status": "completed"
}
```

37. Get Learning Progress

GET `/api/progress/{learningPathId}`

Returns module-by-module progress.

38. Ask Dharma Endpoint

POST `/api/ai/chat`

This is one of the most important APIs.

Request example:

```
{
  "message": "Explain Bhagavad Gita 2.47",
  "locale": "en",
  "conversationId": null,
  "pageContext": {
    "contentType": "verse",
    "contentId": "..."
  }
}
```

39. Ask Dharma Response

Recommended structured response:

```
{
  "conversationId": "...",
  "messageId": "...",
  "answer": {
    "shortAnswer": "...",
    "sections": [
      {
        "type": "explanation",
        "title": "Explanation",
        "content": "..."
      }
    ],
    "citations": [],
    "relatedContent": []
  }
}
```

40. AI Citation Object

Example:

```
{
  "id": "...",
  "label": "Bhagavad Gita 2.47",
  "sourceType": "verse",
  "contentId": "...",
  "url": "/en/scriptures/bhagavad-gita/2/47"
}
```

41. AI Request Validation

Check:

- Message length
- Supported locale
- Valid conversation ID
- Valid page context
- User rate limit
- Content safety
- Authentication state

42. AI Rate Limit Response

If exceeded:

HTTP:

429

Example:

```
{
  "success": false,
  "error": {
    "code": "AI_RATE_LIMIT",
    "message": "Daily AI question limit reached."
  }
}
```

43. AI Conversation List

GET /api/ai/conversations

Authenticated.

Returns user's saved conversations.

44. AI Conversation Detail

GET /api/ai/conversations/{id}

Authenticated.

Must verify that the conversation belongs to the current user.

45. Delete AI Conversation

DELETE `/api/ai/conversations/{id}`

Authenticated.

Prefer soft deletion where appropriate.

46. AI Feedback

POST `/api/ai/feedback`

Request:

```
{
  "messageId": "...",
  "rating": "helpful",
  "reason": null
}
```

Possible rating:

```
helpful
not_helpful
```

47. Report AI Answer

POST `/api/ai/reports`

Request:

```
{
  "messageId": "...",
  "reportType": "wrong_source",
  "description": "..."
}
```

48. Sources API

GET `/api/sources/{id}`

Returns public source information.

Only publicly viewable metadata should be returned.

Do not expose restricted files.

49. Related Content API

GET `/api/related`

Query:

```
contentType
contentId
locale
limit
```

Returns verified related content.

50. Homepage API

Optional aggregation endpoint:

GET `/api/home`

Can return:

- Featured content
- Daily shloka
- Core concepts
- Upcoming festival
- Featured learning path

This reduces many small requests.

51. Daily Shloka

GET `/api/daily-shloka`

Query:

```
locale
date
```

Returns editorially selected or configured verse.

Do not generate a random unverified shloka through AI.

52. Admin Authorization

Every `/api/admin/*` endpoint must verify:

- Authenticated user
- Appropriate role
- Required permissions

Frontend hiding alone is never sufficient.

53. Admin Scriptures

POST `/api/admin/scriptures`

Creates draft scripture.

PATCH `/api/admin/scriptures/{id}`

Updates scripture.

POST `/api/admin/scriptures/{id}/publish`

Publishes after validation and permission checks.

54. Admin Verses

POST `/api/admin/verses`

Create verse.

Required:

- Scripture
 - Section
 - Verse number
 - Sanskrit
 - Source edition
-

55. Verse Validation

Before creating a verse:

- Verify section exists
 - Check duplicate reference
 - Validate Sanskrit field
 - Validate source
 - Set review status
-

56. Admin Translations

POST `/api/admin/translations`

Request:

```
{
  "verseId": "...",
  "locale": "en",
  "translatorId": "...",
  "translationText": "...",
  "sourceId": "...",
  "copyrightStatus": "..."
}
```

57. Admin Commentary

POST `/api/admin/commentaries`

Required:

- Verse
 - Commentator
 - Tradition
 - Source
 - Language
 - Commentary text
-

58. Admin Concepts

Endpoints:

```
POST    /api/admin/concepts
PATCH  /api/admin/concepts/{id}
POST    /api/admin/concepts/{id}/submit-review
POST    /api/admin/concepts/{id}/publish
```

59. Admin Deities

Equivalent CRUD and review endpoints.

60. Admin Mantras

Must require verified source fields before publishing.

The publish endpoint should reject incomplete source metadata when policy requires it.

61. Admin Festivals

Support:

- Festival content
- Regional variations
- Year-specific dates

Date records should be separate from general festival descriptions.

62. Admin Sources

POST `/api/admin/sources`

Creates source record.

PATCH `/api/admin/sources/{id}`

Updates metadata.

POST `/api/admin/sources/{id}/review`

Reviewer action.

63. Content Review API

POST `/api/admin/reviews`

Request:

```
{
  "contentType": "concept",
  "contentId": "...",
  "reviewType": "scripture",
  "status": "approved",
  "notes": "..."
}
```

64. Content Publish Rules

Publishing should happen through a dedicated server-side operation.

Do not simply allow editors to set:

```
status = published
```

directly from the browser.

The publish service should validate required conditions.

65. Example Publish Checks

Before publishing a verse explanation:

- Source exists
 - Required reviews completed
 - Locale is valid
 - No missing references
 - Content is not archived
-

66. Content Reports

POST `/api/reports/content`

Public or authenticated.

Request:

```
{
  "contentType": "verse",
  "contentId": "...",
  "reportType": "incorrect_translation",
  "description": "..."
}
```

Rate limiting should prevent spam.

67. Admin Reports

GET `/api/admin/reports`

Filters:

```
status
type
contentType
```

68. Report Resolution

PATCH `/api/admin/reports/{id}`

Example:

```
{
  "status": "resolved",
  "resolutionNote": "Translation corrected."
}
```

69. Media Uploads

POST `/api/admin/media/upload`

Use secure signed upload workflows when possible.

Validate:

- File type
 - File size
 - User permission
-

70. Supported Initial Media

V1:

- Images
- Basic documents where necessary

Audio can be added later.

71. API Pagination

Use consistent pagination.

Recommended:

```
page
limit
```

Example response:

```
{
  "items": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 134,
    "totalPages": 7
  }
}
```

For very large datasets later, cursor pagination may be preferable.

72. API Sorting

Where relevant:

```
sort
order
```

Examples:

```
title
publishedAt
date
popularity
```

Do not expose arbitrary database column sorting.

73. Locale Handling

Endpoints should explicitly accept or infer locale.

Priority:

1. Explicit request locale
2. User preference
3. Site locale
4. Default `en`

Do not mix Hindi and English unexpectedly.

74. API Caching

Safe public endpoints may be cached.

Good candidates:

- Scripture overview
- Concepts
- Deities
- Mantras
- Festival descriptions

Do not cache personalized responses publicly.

75. AI Caching

Potentially cache only safe, non-personalized questions.

Avoid caching:

- User-specific messages
 - Private conversation context
 - Account information
-

76. Rate Limiting

Apply stronger rate limits to:

- AI
 - Login attempts
 - Reports
 - Search suggestions
 - Admin sensitive actions
-

77. Idempotency

Use idempotent behavior where appropriate.

Example:

Bookmarking the same verse twice should not create duplicates.

78. Error Codes

Recommended internal codes:

```
AUTH_REQUIRED
FORBIDDEN
NOT_FOUND
VALIDATION_ERROR
DUPLICATE_RECORD
RATE_LIMITED
SOURCE_NOT_VERIFIED
CONTENT_NOT_PUBLISHABLE
AI_RATE_LIMIT
AI_PROVIDER_ERROR
```

```
AI_RETRIEVAL_FAILED
INTERNAL_ERROR
```

79. API Security

Requirements:

- Authentication
- Authorization
- Input validation
- Rate limiting
- Secure cookies
- No secrets in client responses
- Proper CORS if external clients are added
- Audit sensitive admin actions

80. Service Layer

Route handlers should remain thin.

Recommended structure:

```
Route Handler
↓
Validation
↓
Service
↓
Repository / Database
```

Example:

```
POST /api/bookmarks
↓
bookmarkService.add()
↓
bookmarkRepository.create()
```

This helps future mobile and testing.

81. Recommended Backend Modules

```
auth
users
content
scriptures
verses
translations
commentaries
concepts
deities
mantras
festivals
search
bookmarks
learning
sources
reviews
ai
admin
media
```

82. Public vs Private API

Public:

- Published content
- Search
- Source metadata
- Limited AI

Private:

- Bookmarks
- Progress
- History
- Profile
- Saved AI chats

Admin:

- Editing
- Review

- Publishing
 - User role management
-

83. Future Mobile Support

Mobile applications should be able to use the same core service functionality.

Do not make critical business logic dependent on React components.

84. API Versioning

V1 does not need elaborate API versioning.

If external/public APIs are introduced later, use:

```
/api/v1/
```

For now, internal `/api/` routes are sufficient.

85. API Documentation

During implementation, maintain machine-readable/API documentation where useful.

Possible:

- OpenAPI specification
- Generated endpoint documentation

At minimum, maintain request/response types in TypeScript.

86. Type Sharing

Share TypeScript types or schemas between:

- Frontend
- Route validation
- Backend services

Zod schemas can help derive types and reduce duplication.

87. Example Verse Retrieval Flow

```
Browser requests verse
↓
GET /api/verses/{id}
↓
Validate ID
↓
Load published verse
↓
Load translations
↓
Load commentary
↓
Load sources
↓
Load related content
↓
Return structured response
```

88. Example Bookmark Flow

```
User presses Bookmark
↓
POST /api/bookmarks
↓
Authenticate
↓
Validate content
↓
Check duplicate
↓
Create bookmark
↓
Return saved state
```

89. Example Ask Dharma Flow

```
User asks question
↓
POST /api/ai/chat
↓
Authenticate or anonymous session
↓
Check usage limit
↓
Validate message
↓
Detect language
↓
Classify question
↓
Retrieve sources
↓
Generate answer
↓
Validate citations
↓
Store conversation if applicable
↓
Return structured answer
```

90. Example Admin Publishing Flow

```
Editor creates draft
↓
Source check
↓
Reviewer approval
↓
POST /publish
↓
Backend verifies requirements
↓
Status updated
↓
Search index updated
↓
```

AI knowledge index updated

↓

Cache invalidated

91. AI Index Synchronization

When verified content is published:

- Generate/update embedding
- Add it to production retrieval
- Update search document
- Remove old chunk if replaced

When content becomes unverified or archived:

Remove it from AI retrieval.

92. Search Index Synchronization

Publishing content should update search indexes automatically.

The system should not require manual reindexing for ordinary edits.

93. Cache Invalidation

When content changes:

Invalidate:

- Detail page cache
- Relevant lists
- Search document
- Related content cache

Avoid serving old religious content after a correction.

94. Logging

Log:

- API errors
- Slow requests
- AI provider failures
- Admin publication events

Do not log secrets or unnecessary private chat content.

95. V1 Required APIs

Required for initial launch:

```
GET /api/home

GET /api/scriptures
GET /api/scriptures/{slug}
GET /api/verses/{id}

GET /api/concepts
GET /api/concepts/{slug}

GET /api/deities
GET /api/deities/{slug}

GET /api/mantras
GET /api/mantras/{slug}

GET /api/festivals
GET /api/festivals/{slug}

GET /api/search
GET /api/search/suggestions

GET /api/me
PATCH /api/me/profile
PATCH /api/me/preferences

GET /api/bookmarks
POST /api/bookmarks
DELETE /api/bookmarks/{id}
```

```
GET /api/progress/{learningPathId}
POST /api/progress

POST /api/ai/chat
GET /api/ai/conversations
GET /api/ai/conversations/{id}
DELETE /api/ai/conversations/{id}
POST /api/ai/feedback

POST /api/reports/content
```

Admin APIs should be built as the content workflow is introduced.

96. Acceptance Criteria

The API layer is ready when:

- All inputs are validated.
 - Public content returns only published records.
 - User data is protected.
 - Bookmarks cannot be duplicated.
 - Progress is user-specific.
 - AI rate limiting works.
 - AI citations resolve to real content.
 - Admin permissions are server-enforced.
 - Search supports English and Hindi.
 - Error responses are consistent.
 - Mobile apps can later reuse core services.
-

97. Status

API Specification:

Initial Version Complete

The project now has defined:

- Public content APIs
- Search
- Authentication-aware APIs
- Bookmarks
- Learning progress

- AI endpoints
- Citations
- Feedback
- Admin workflows
- Publishing
- Index synchronization
- Security
- Validation
- Error standards